



Python Practice Tasks

Lists, List Manipulation, and Tuples

Difficulty Level

Medium difficulty exercises designed to strengthen practical understanding of Python collections, iteration, list methods, tuple access, and basic calculations.

Task 1	Student Grade Manager	Lists and calculations
Task 2	Shopping Cart Manager	List manipulation methods
Task 3	Employee Information	Tuples and nested data

Programming with AI | Python Fundamentals

Task 1: Student Grade Manager

Lists

Objective

Create a program that manages student grades using a Python list.

Requirements

- Ask the user to enter 5 student grades.
- Store the grades in a list.
- Display the highest grade, lowest grade, and average grade.
- Ask the user for one more grade and append it to the list.
- Sort the list in ascending order.
- Print the final sorted list.

Example Output

```
Enter grade 1: 80
Enter grade 2: 90
Enter grade 3: 75
Enter grade 4: 95
Enter grade 5: 88

Highest grade: 95
Lowest grade: 75
Average: 85.6

Enter another grade: 92

Sorted grades:
[75, 80, 88, 90, 92, 95]
```

Programming Focus

Use a list, input conversion, `append()`, `sort()`, `max()`, `min()`, `sum()`, `len()`, and formatted output.

Task 2: Shopping Cart Manager

List Manipulation

Objective

Practice manipulating lists by using common Python list methods.

Requirements

- Create an empty shopping cart list.
- Ask the user to enter 5 products.
- Display the shopping cart.
- Ask the user which product they want to remove.
- Remove the product only if it exists in the list.
- Ask for a new product and insert it at index 2.
- Sort the list alphabetically.
- Print the total number of products, first product, last product, and final shopping cart.

Example Output

```
Shopping cart:
['Milk', 'Bread', 'Eggs', 'Juice', 'Rice']

Enter a product to remove:
Eggs

Enter a new product:
Butter

Final cart:
['Bread', 'Butter', 'Juice', 'Milk', 'Rice']

Total products: 5
First: Bread
Last: Rice
```

Programming Focus

Use `append()`, `remove()`, `insert()`, `sort()`, membership checking with `in`, indexing, and `len()`.

Task 3: Employee Information

Tuples and Nested Data Structures

Objective

Practice creating, storing, accessing, and analyzing tuples.

Tuple Format

```
(ID, Name, Department, Salary)
```

Requirements

- Create three employee tuples.
- Store the three tuples inside a list.
- Display all employee information.
- Print only the names of all employees.
- Find and display the employee with the highest salary.
- Calculate and print the average salary.

Example Data and Output

```
Employee 1:
(101, 'John', 'IT', 1200)

Employee 2:
(102, 'Anna', 'HR', 950)

Employee 3:
(103, 'David', 'Finance', 1400)

Employee Names:
John
Anna
David

Highest Salary:
David - 1400

Average Salary:
1183.33
```

Programming Focus

Use tuples, a list of tuples, loops, indexing, max logic, salary totals, and average calculation.

Learning Outcomes

After completing these tasks, students should be able to:

- Create and use Python lists.
- Apply list methods such as `append()`, `insert()`, `remove()`, and `sort()`.
- Loop through lists and process user input.
- Create and access tuples.
- Work with nested data structures such as a list of tuples.
- Perform calculations using maximum, minimum, totals, and averages.
- Format program output in a readable way.

Submission Note

Students should submit one Python file for each task. Use meaningful variable names and include brief comments where necessary.

